
EMBEDDED HACKING

Reverse Engineering the Raspberry Pi Pico 2

GDB Hardware Debugging Tutorial: Reverse Engineering the Stripped RP2350 Image

ARM Cortex-M33 • GDB • Ghidra

September 27, 2026

For educational and defensive security research only

0. Cold Open

The board is powered. The blade is turning. And none of it is true to you yet, because you have not seen it with your own eyes.

Every claim you will make about this machine has to survive one test: did you watch it happen? Not did the decompiler suggest it. Not did the datasheet imply it. Did you stop the core, read the register, and see the number with your own eyes.

The Debug Probe is the only honest witness in the room. It reaches through SWD into the silicon and pulls out the truth at 5,000 kHz while the rest of the world argues. It does not care what you believe. It does not care what Buddy answered. It reports.

A register is not an opinion. A clock divider is not a narrative. `SM0_CLKDIV = 0x07A12000` is not a talking point. It is 1953.125, and 1953.125 is the reason the sky is readable at all.

Anyone can generate an explanation. You are here to *verify* one, bit by bit, on live silicon, and to sign your name to it.

Think, then verify.

1. Executive Summary

This tutorial reverse engineers the **stripped** `0x0011a_cb.bin` on live silicon using a Raspberry Pi Debug Probe, OpenOCD, and GNU GDB. There are **no symbols**, no `main`, no variable names, nothing. You set breakpoints by **address**, read raw memory, and let the hardware tell you the truth.

Everything shown was captured from the real target and is reproducible.

```
+-----+
|                GDB HARDWARE DEBUG SIGNAL CHAIN                |
+-----+
| HOST macOS -> USB -> Debug Probe -> SWD -> RP2350B Cortex-M33 |
| OpenOCD :3333 <----- SWD + UART bridge -----> GP0/GP1 115200 |
+-----+
```

The philosophy: an offline model can guess what a stripped image does. It cannot read the live registers, watch the parser, or verify a patch. **Think, then verify.**

2. Prerequisites

Component	Value
Target	Raspberry Pi Pico 2 (RP2350B)
Image	<code>0x0011a_cb.bin</code> (raw flash image, base <code>0x10000000</code>)
Probe	Raspberry Pi Debug Probe (CMSIS-DAP v2)
Architecture	ARM:LE:32:Cortex (ARMv8-M / Cortex-M33, Thumb-2)

The `.bin` is the stripped image. `file offset = address, 0x10000000`.

2.1 Tool Paths Per Host (macOS, Linux, Windows)

The probe, OpenOCD, and GDB behave identically on every platform; only the paths and the shell differ. Pick your host.

Tool	macOS	Linux	Windows
OpenOCD	<code>~/ .pico-sdk/openocd/0.12.0+dev/openocd</code>	<code>\$HOME/.pico-sdk/openocd/0.12.0+dev/openocd</code>	<code>%USERPROFILE%\ .pico-sdk\openocd\0.12.0+dev\openocd.exe</code>

Tool	macOS	Linux	Windows
OpenOCD scripts	<code>~/pico-sdk/openocd/0.12.0+dev/scripts</code>	<code>\$HOME/.pico-sdk/openocd/0.12.0+dev/scripts</code>	<code>%USERPROFILE%\pico-sdk\openocd\0.12.0+dev\scripts</code>
GDB	<code>~/pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb</code>	<code>\$HOME/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb</code>	<code>%USERPROFILE%\pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe</code>
Serial port	<code>/dev/tty.usbmodem*</code>	<code>/dev/ttyACM*</code>	COMx (Device Manager)

Install the tools if you do not have them:

- macOS: `brew install --cask gcc-arm-embedded` for the toolchain, then `brew install open-ocd`, or run the Raspberry Pi `pico-setup` script, which places everything under `~/pico-sdk`.
- Linux: install `gcc-arm-none-eabi` and `openocd` from your package manager, or run the Raspberry Pi `pico-setup` script under `~/pico-sdk`.
- Windows: install the Raspberry Pi Pico VS Code extension or the official Windows installer; both place the toolchain under `%USERPROFILE%\pico-sdk`. Use PowerShell for every command below.

Serial console to the debug UART, per host:

```
# macOS
screen /dev/tty.usbmodem* 115200
# Linux
screen /dev/ttyACM* 115200
```

```
# Windows
putty -serial COMx -sercfg 115200,8,n,1
```

3. Flash the Stripped Image

Because there are no object headers, you flash the raw bytes at the flash base explicitly.

macOS and Linux:

```
OCD=~/pico-sdk/openocd/0.12.0+dev/openocd
SCR=~/pico-sdk/openocd/0.12.0+dev/scripts
$OCD -s "$SCR" -f interface/cmsis-dap.cfg -f target/rp2350.cfg \
  -c "adapter speed 5000" \
  -c "program 0x0011a_cb.bin 0x10000000 verify reset exit"
```

Windows PowerShell:

```
$OCD = "$env:USERPROFILE\pico-sdk\openocd\0.12.0+dev\openocd.exe"
$SCR = "$env:USERPROFILE\pico-sdk\openocd\0.12.0+dev\scripts"
& $OCD -s "$SCR" -f interface/cmsis-dap.cfg -f target/rp2350.cfg `
  -c "adapter speed 5000" `
  -c "program 0x0011a_cb.bin 0x10000000 verify reset exit"
```

Expected on every host:

```
** Programming Started **
** Programming Finished **
** Verified OK **
** Resetting Target **
```

4. Attach GDB With No Symbols

Start the OpenOCD GDB server in one terminal, then attach **without** an executable in another.

macOS and Linux:

```
OCD=~/.pico-sdk/openocd/0.12.0+dev/openocd
SCR=~/.pico-sdk/openocd/0.12.0+dev/scripts
$OCD -s "$SCR" -f interface/cmsis-dap.cfg -f target/rp2350.cfg -c "adapter speed 5000" &

GDB=~/.pico-sdk/toolchain/14_2_Rel1/bin/arm-none-eabi-gdb
$GDB -q
(gdb) target extended-remote localhost:3333
(gdb) monitor reset halt
```

Windows PowerShell:

```
$OCD = "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\openocd.exe"
$SCR = "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\scripts"
Start-Process $OCD -ArgumentList @("-s","$SCR","-f","interface/cmsis-dap.cfg","-f","target/rp2350.cfg","-c","adapter speed 5000")

$GDB = "$env:USERPROFILE\.pico-sdk\toolchain\14_2_Rel1\bin\arm-none-eabi-gdb.exe"
& $GDB -q
(gdb) target extended-remote localhost:3333
(gdb) monitor reset halt
```

`bt` and `break main` are useless: there are no symbols. Every stop is an address. This is the stripped-image reality.

5. Break at the Reset Handler's Destination

From static analysis (see the Ghidra tutorial) we know the code entry `main` begins at `0x10000234`. Break there by address:

```
(gdb) break *0x10000234
(gdb) continue
Thread 1 hit Breakpoint 1, 0x10000234 in ?? ()

(gdb) info registers r0 r1 r2 r3 sp lr pc xpsr
r0  0x0 0
r1  0x10000235 268436021
r2  0x80808080 -2139062144
r3  0xe000ed08 -536810232
sp  0x20082000
lr  0x1000018f
pc  0x10000234
xpsr 0x69000000
```

Backtrace shows raw addresses only:

```
(gdb) bt
#0 0x10000234 in ?? ()
```

6. The Decoy And The Encrypted Target (No Symbols Needed)

The firmware carries **two** kinds of coordinate, and they are not the same kind of thing.

The decoy (plaintext). A plaintext pair sits at `0x1000A098` (lat) / `0x1000A090` (lon): `+38.840280` / `-77.428890`. The beacon broadcasts it and a byte scanner grabs it. It is the lie.

DECOY WAYPOINT (PLAINTEXT, THE LIE)			
0x1000A098	CF BD 87 4B 8E 6B 43 40	double	+38.840280
0x1000A090	36 E5 0A EF 72 5B 53 C0	double	-77.428890

The target (AES-encrypted). The real waypoint is never stored as a double:

```
(gdb) x/16bx 0x10009D90
0x10009D90: 0x56 0x45 0x43 0x54 0x4f 0x52 0x31 0x31 # "VECTOR11"
0x10009D98: 0x41 0x45 0x53 0x4b 0x45 0x59 0x21 0x21 # "AESKEY!!"
(gdb) x/2gx 0x10009DA4
0x10009DA4: 0x7aea23c0c2ac4f20 0xcaef2311710f933a # ciphertext block
```

`init_navigation @ 0x10000C2C` decrypts each word with the key into RAM at `TARGET_LAT 0x20000D00` / `TARGET_LON 0x20000D08`. Read the reconstructed truth after boot:

```
(gdb) x/2gx 0x20000d00
0x20000d00: 0x404370e368f08462 0xc0535cd1633482bf
```

Decode it: +38.881940 / -77.450280 (NRO HQ). The decoy says Centreville. The target says Chantilly. Same firmware. One of them is a lie.

7. Prove the Peripherals From Registers

Break at `send_telemetry @ 0x10000C7C` (called once per acquisition tick) so the initialisers have already run:

```
(gdb) break *0x10000c7c
(gdb) continue
Thread 1 hit Breakpoint 2, 0x10000c7c in ?? ()

(gdb) x/2xw 0x50200000
0x50200000: 0x00000001 0x0f010e01 # PI00_CTRL      PI00_FSTAT
(gdb) x/6xw 0x502000c8
0x502000c8: 0x07a12000 0x0701fc00 # SM0_CLKDIV      SM0_EXECCTRL
0x502000d0: 0x800c0000 0x00000018 # SM0_SHIFTCTRL   SM0_ADDR
0x502000d8: 0x00002020 0x00038000 # SM0_INSTR       SM0_PINCTRL
```

```
(gdb) x/4xw 0x40078024
0x40078024: 0x000003d0 0x00000024 0x00000070 0x00000301 # UART1 LoRa
(gdb) x/4xw 0x40070024
0x40070024: 0x00000051 0x00000018 0x00000070 0x00000301 # UART0 debug
```

PI00 SM0 CLOCK DIVIDER (RP2350 @ 150 MHz, 8 cycles/bit)	
SM0_CLKDIV = 0x07A12000	= 1953 + 32/256 = 1953.125
f _{sm} = 150,000,000 / 1953.125	= 76,800 Hz = 9600 x 8

UART baud uses `IBRD` and a 6-bit `FBRD`:

$$baud = \frac{f_{clk}}{16 \times (IBRD + FBRD/64)}$$

UART1: $976 + 36/64 = 976.5625 \Rightarrow 150,000,000/15625 = 9600$. UART0: $81 + 24/64 = 81.375 \Rightarrow 150,000,000/1302 \approx 115200$.

8. Watch the NMEA Parser Prove Itself

Static analysis identifies the parser's static index at `0x200010f4` (the `.bss` symbol `idx.1`) and the 96-byte NMEA buffer at `0x20001044` (`buf.0`). Watch the index:

```
(gdb) watch *(int*)0x200010f4
(gdb) continue
Hardware watchpoint 3: *(int*)0x200010f4
Old value = 0
New value = 1
0x10000458 in ?? ()

(gdb) bt
#0  0x10000458 in ?? ()
#1  0x1000027c in ?? ()

(gdb) x/32cb 0x20001044
0x20001044: 36 '$' 71 'G' 80 'P' 71 'G' 76 'L' 76 'L' ...
# => "$GPGLL,,,,,220653.00,V,N*4A"
```

At reset the buffer and index are both zero; these bytes appear only after a sentence is parsed off the wire.

The `V` says there is **no fix yet**, not a parser bug, not a UART fault. Only the wire can tell you that.

9. The Actuators: `release_payload @ 0x10000bfc`

The Ghidra analysis (companion tutorial) shows `release_payload` drives GP16, GP17 and GP18 high through the RP2350 GPIO coprocessor interface. It is reached by a `b.w` tail call from `navigate_to_target` at `0x10000d18`, so there is no return frame; break at its entry and step the writes:

```
(gdb) break *0x10000bfc
(gdb) continue
Thread 1 hit Breakpoint 4, 0x10000bfc in ?? ()

(gdb) x/9i $pc
0x10000bfc: push {r3, lr}
0x10000bfe: movs r2, #16
0x10000c00: mov.w r3, #1
0x10000c04: mcrr 0, 4, r2, r3, cr0 # GPIO16 = 1
0x10000c08: movs r2, #17
0x10000c0a: mcrr 0, 4, r2, r3, cr0 # GPIO17 = 1
0x10000c0e: movs r2, #18
0x10000c10: mcrr 0, 4, r2, r3, cr0 # GPIO18 = 1
```

After the three writes, the SIO register reads `0x00070000` (bits 16, 17, 18). The RP2350 exposes GPIO through the coprocessor (`mcrr p0, #4, ...`), not a plain store, a fact only the bench reveals.

10. Reproducibility Checklist

1. Flash `0x0011a_cb.bin` at `0x10000000`; Verified OK.
2. Attach GDB with **no** symbol file.
3. `break *0x10000234`; PC lands exactly there.
4. `x/4xw 0x20000d00 -> 68f08462 404370e3 633482bf c0535cd1`.
5. `x/6xw 0x502000c8 -> SM0_CLKDIV = 0x07A12000`.
6. `watch *(int*)0x200010f4` trips only when NMEA arrives.
7. `x/9i 0x10000bfc` shows the `mcrr` GPIO coprocessor writes.

If a single value differs, you are not on the image you think you are. That check is the job an offline model cannot do.

11. Why Buddy Fails Here

Even a model trained on ARM cannot read `SM0_CLKDIV`, watch `idx.0`, decode *your* randomized literal, or verify a patch by flashing it. It generates plausible text; the bench generates truth. **Think, then verify.**

Appendix A. Deep GDB Step-Through

A.1 Start the server and attach

macOS:

```
OCD=~/.pico-sdk/openocd/0.12.0+dev/openocd
SCR=~/.pico-sdk/openocd/0.12.0+dev/scripts
$OCD -s "$SCR" -f interface/cmsis-dap.cfg -f target/rp2350.cfg -c "adapter speed 5000"
```

Linux:

```
OCD=$HOME/.pico-sdk/openocd/0.12.0+dev/openocd
SCR=$HOME/.pico-sdk/openocd/0.12.0+dev/scripts
$OCD -s "$SCR" -f interface/cmsis-dap.cfg -f target/rp2350.cfg -c "adapter speed 5000"
```

Windows PowerShell:

```
$OCD = "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\openocd.exe"
$SCR = "$env:USERPROFILE\.pico-sdk\openocd\0.12.0+dev\scripts"
& $OCD -s "$SCR" -f interface/cmsis-dap.cfg -f target/rp2350.cfg -c "adapter speed 5000"
```

Then attach from GDB (identical on all hosts). There are no symbols, so break by address, not by name:

```
target extended-remote localhost:3333
monitor reset halt
break *0x10000234
continue
```

A.2 Read the vector table and the key material

```
x/2xw 0x10000000      # SP and reset handler
x/16bx 0x10000D90     # AES key
x/16bx 0x10000DA4     # ciphertext block
x/2gx 0x20000D00      # decrypted target after boot
```

A.3 Break the AES routine

```
break *0x10000E5C
continue
x/4i $pc
stepi
stepi
```

`aes128_ecb_decrypt_block` runs once, early, inside `init_navigation`.

A.4 Watch the decoy go out

```
break *0x10000C7C
continue
info registers r0 r1 r2 r3
```

`send_telemetry` is called with the decoy while the GNSS has no fix.

A.5 The tool trap

On a stripped target a floating point cast can byte-swap. Trust the raw read:

```
x/1gx 0x20000D00 # then decode it yourself
```